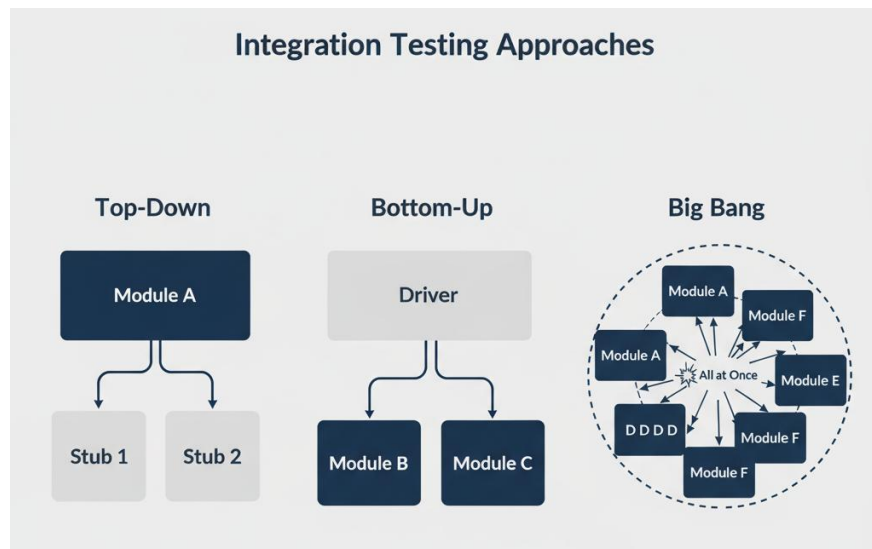


UNIT IV: Testing Strategies

Q1. What is Integration testing? Explain types of it.

Integration Testing is the second level of testing, performed after Unit Testing. Its purpose is to test the **interaction and interfaces** between two or more software modules (units) that have already been unit tested. The main goal is to find defects that arise when modules are combined. For example, one module might be sending data in a format that the other module does not understand.



Types of Integration Testing: There are two main approaches to integration testing:

1. Big Bang Approach: In this approach, all or most of the modules are combined together at once and tested as a single system.

- **Process:** All modules are integrated, and then the testing is carried out.
- **Advantages:** It is simple and good for very small systems.
- **Disadvantages:** It is very difficult to find the root cause of a bug. If a test fails, it is hard to know which module or interface caused the problem.

2. Incremental Approach: This is the preferred method. Modules are added one by one (incrementally), and the system is tested at each step. This makes it much easier to find bugs. This approach has 3 types:

- **a) Top-Down Integration:** Testing starts from the main (top-level) module and moves downwards. If a lower-level module is not yet ready, a temporary dummy module called a **"Stub"** is used to simulate its behavior.
 - **Example:** Test the "Main Menu" first. Use stubs for "Login," "Profile," and "Settings."
- **b) Bottom-Up Integration:** Testing starts from the lowest-level modules (e.g., utility functions) and moves upwards. To test these lower modules, a temporary module called a **"Driver"** is created to call the module and pass it test data.
 - **Example:** Test the "Database Connection" module first using a Driver, then test the "Login" module that depends on it.
- **c) Sandwich (or Hybrid) Integration:** This approach combines both Top-Down and Bottom-Up testing. The top-level modules are tested with stubs, and the low-level modules are tested with drivers. The "middle" layer is tested when these two parts meet.

Q2. Explain System testing.

System Testing is the third level of testing, performed after Integration Testing. It is the first time the **complete and fully integrated** software application is tested as a whole.

The main purpose of System Testing is to verify that the entire system meets all the specified **functional and non-functional requirements** defined in the requirements document (e.g., Software Requirement Specification - SRS).

Key Characteristics of System Testing:

- **Black-Box Testing:** It is a form of Black-Box testing, meaning the tester does not need to know the internal code or logic. The focus is on the expected behavior of the system from an end-user's perspective.
- **End-to-End Testing:** It tests the complete flow of the application from start to finish. For example, in an e-commerce site, a system test would check the entire process from "searching for a product" to "adding to cart" and "completing the checkout."
- **Covers All Requirements:** System testing is a broad category that includes many different types of tests to check all requirements:
 - **Functional Testing:** Does the system perform all its required functions?
 - **Performance Testing:** Is the system fast and stable under load?
 - **Security Testing:** Is the system secure from attacks?
 - **Usability Testing:** Is the system easy to use?
 - **Reliability Testing:** Does the system work without crashing?
 - **Compatibility Testing:** Does the system work on different browsers, operating systems, etc.?

Goal: The goal of system testing is to find any defects in the complete system before it is handed over to the client or end-users for Acceptance Testing.

Q3. Explain Acceptance testing.

Acceptance Testing (also known as User Acceptance Testing or UAT) is the **final level** of software testing. It is performed after System Testing and before the software is released to the public.

Main Purpose: The primary goal of Acceptance Testing is to determine if the software is "acceptable" for delivery and meets the **business needs and requirements of the client or end-user**. It is not focused on finding deep technical bugs, but rather on ensuring the software is "fit for purpose."

Who Performs It: This is the key differentiator. Acceptance testing is typically performed by the **client, end-users, or stakeholders**, not by the development or QA team.

Types of Acceptance Testing: There are two main types of acceptance testing:

1. Alpha Testing: This is testing done *internally* by the organization that developed the software, but by a team that was not involved in the project (e.g., internal QA, support staff, or other employees).

- **Environment:** It is performed in a controlled "test" environment at the developer's site.
- **Goal:** To find and fix bugs before the software is shown to external users.

2. Beta Testing: This is testing done *externally* by a limited number of *real end-users* in the "real world." This is often called a "beta version" of the software.

- **Environment:** It is performed in the user's own environment (on their own computers, with their own data).
 - **Goal:** To get feedback from real users on performance, usability, and compatibility, and to find bugs that were missed in the controlled Alpha and System testing phases.
-

Q4. Explain Regression testing.

Regression Testing is the process of re-testing a software application after a **change** has been made, to ensure that the new change has not "regressed" (broken) any of the existing, previously working features.

When is it Performed? Regression testing is performed every time the source code is modified. This includes:

- When a new feature is added.
- When a bug is fixed.
- When performance is enhanced.
- When the system's environment (e.g., database) is changed.

Main Goal: The goal is not to test the new feature itself (that is done in other tests). The goal is to make sure that the old features still work correctly and that the new change did not have any unintended side effects.

How it is Performed:

- Testers use a collection of existing test cases called a "**Regression Test Suite**".
- This suite contains test cases that cover the main functionalities of the application.
- After a code change, the team re-runs this entire test suite. If any test case in this suite fails, it indicates a regression.

Example:

- **Application:** A calculator app.
- **Existing Features:** Addition, Subtraction, Multiplication all work perfectly.
- **New Change:** A developer adds a new "Division" feature.
- **Regression Test:** The QA team tests the new "Division" feature (this is regular testing). Then, they perform **regression testing** by re-running the tests for Addition, Subtraction, and Multiplication to make sure the new Division code didn't accidentally break them.

Because regression tests are run very frequently, they are the most common tests to be **automated**.

Q5. Explain Ad hoc Testing.

Ad hoc Testing is an **informal, unstructured** testing method that is performed without any test plan, test cases, or documentation.

It is also commonly known as "**Monkey Testing**" or "**Exploratory Testing**" (though exploratory testing is often slightly more structured).

Key Characteristics:

- **No Planning:** The tester does not follow any pre-defined steps.

- **Improvisation:** The tester relies entirely on their **intuition, experience, and creativity** to find bugs.
- **Goal:** The main goal of ad hoc testing is to "break the system" and find bugs that were likely missed by the formal, planned testing (which follows a strict script).
- **Process:** The tester tries random inputs, clicks buttons in an unexpected order, and explores the application's features to see if anything breaks.

Example: In a web form, a formal test case might check "Enter valid email" and "Enter invalid email." An ad hoc tester might try:

- Pasting 5000 characters into the field.
- Entering an email with special characters (!@#\$%^).
- Leaving the field blank and trying to submit the form by pressing "Enter" instead of clicking the "Submit" button.
- Trying to use the "Back" button after submitting.

When is it used? Ad hoc testing is usually performed after the formal testing is completed, as a final check to catch any remaining, hidden bugs. It is most effective when done by an experienced tester who knows the application well.

Q6. Explain GUI Testing.

GUI (Graphical User Interface) Testing is a type of testing that focuses on the **visual aspects** ("look and feel") and **functional behavior** of the software's user interface.

The goal is to ensure that the application's interface is user-friendly, easy to use, and works as expected from a user's point of view.

What does GUI Testing check? GUI testing involves checking several key components:

- 1. Visual Design and Layout:**
 - Are all buttons, text boxes, labels, and images aligned correctly?
 - Do the colors, fonts, and images match the design specifications?
 - Does the interface look correct on different screen sizes and resolutions (responsiveness)?
- 2. Functionality of Controls:**
 - Do buttons work when clicked?
 - Can the user type data into text fields?
 - Do drop-down lists expand and show the correct options?
 - Do links navigate to the correct page?
 - Do menus and checkboxes work correctly?
- 3. Usability and Navigation:**
 - Is the application easy to navigate?
 - Is the workflow logical and intuitive for the user?
 - Are all the controls and labels easy to understand?
- 4. Error Messages:**
 - When the user makes a mistake (e.g., enters an invalid email), is the error message clear, helpful, and user-friendly?
- 5. Consistency:**
 - Does the application have a consistent "look and feel" across all its screens? (e.g., is the "Save" button always in the same place and the same color?)

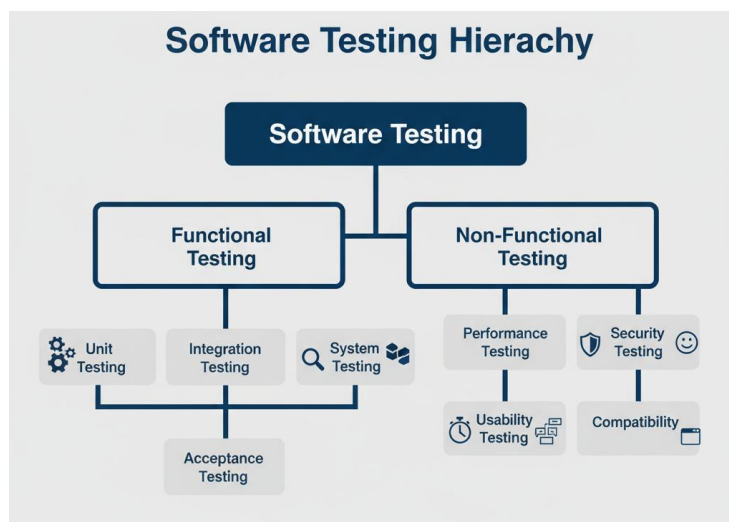
Example: For a login screen, GUI testing would check:

- If the "Username" and "Password" text boxes are aligned.
- If the "Login" button is the correct color and size.
- If the user can type in the text boxes.
- If a pop-up appears when the user clicks the "Forgot Password" link.
- If an error message like "Username is required" appears if the user clicks "Login" with the field empty.

Q7. What are different types of testing.

Software testing can be categorized in many different ways, based on the testing objective, the phase of development, or the technique used.

Here are the main categories and types of testing:



1. Testing by Levels (SDLC Phases): This describes the *order* in which testing is performed.

- **Unit Testing:** Testing individual modules or components of code in isolation (usually done by developers).
- **Integration Testing:** Testing the interfaces and interaction between two or more integrated modules.
- **System Testing:** Testing the complete, integrated application to verify it meets all requirements.
- **Acceptance Testing (UAT):** Final testing by the client or end-users to check if the software is "acceptable" for release.

2. Testing by Technique (Code Visibility): This describes *how* the tests are designed.

- **White Box Testing:** The tester knows the internal code structure. Testing is based on code paths, branches, and statements. (e.g., Unit Testing).
- **Black Box Testing:** The tester does *not* know the internal code. Testing is based on requirements and functionality. (e.g., System Testing, Acceptance Testing).
- **Grey Box Testing:** A combination of both. The tester has partial knowledge of the internal code (e.g., to design better test data).

3. Testing by Purpose (What is being checked): This is the largest category, which includes:

A) Functional Testing: Tests the *features* of the software ("what it does").

- **Examples:** Smoke Testing, Sanity Testing, Regression Testing, Use Case Testing.

B) Non-Functional Testing: Tests the *quality* of the software ("how well it works").

- **Examples:**
 - **Performance Testing:** Checks the speed, stability, and scalability (e.g., Load Testing, Stress Testing).
 - **Security Testing:** Checks for vulnerabilities and ensures data is safe.
 - **Usability Testing:** Checks if the software is easy to use.
 - **Compatibility Testing:** Checks if the software works on different browsers, OS, and devices.
 - **Reliability Testing:** Checks if the software is stable and can run without crashing for a long time.
-

Q8. Explain functional & non functional testing.

Functional and Non-Functional testing are two main categories of System Testing. They test different aspects of the software.

Functional Testing: This type of testing verifies *what* the software does.

- **Main Goal:** Its goal is to test the **features and functions** of the application. It checks if the software's behavior matches the functional requirements (e.g., "The user must be able to log in").
- **Focus:** It focuses on specific actions and outputs.
 - "When I enter valid data and click 'Save', is the data saved?"
 - "When I click the 'Print' button, does the document print?"
- **Test Result:** The result is a simple Pass/Fail. The feature either works or it doesn't.
- **Examples of Functional Tests:**
 - **Unit Testing**
 - **Integration Testing**
 - **Smoke Testing**
 - **System Testing** (the functional parts)
 - **Regression Testing**

Non-Functional Testing: This type of testing verifies *how well* the software works.

- **Main Goal:** Its goal is to test the **quality attributes** (also called "non-functional requirements") of the application, such as its performance, security, and usability.
- **Focus:** It focuses on the overall quality and user experience.
 - "How *fast* is the 'Save' operation?"
 - "How *many* users can log in at the same time before the system crashes?"
 - "How *easy* is it to find the 'Print' button?"
 - "Is the system *secure* from hackers?"
- **Test Result:** The result is not just Pass/Fail, but a measurement (e.g., "The page loaded in 3.5 seconds").
- **Examples of Non-Functional Tests:**
 - **Performance Testing** (Load, Stress, Volume Testing)
 - **Security Testing**
 - **Usability Testing**
 - **Compatibility Testing** (on different browsers/OS)
 - **Reliability Testing**